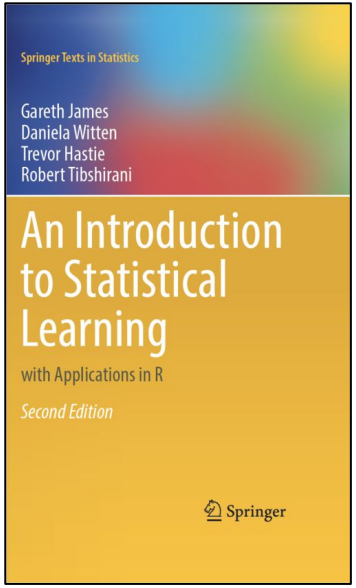
Overfitting and the Bias-Variance Decomposition

Recommended Reading:

An Introduction to Statistical Learning, Sections 2.2.2 and 6.2

Download it for free here:
https://www.statlearning.com/

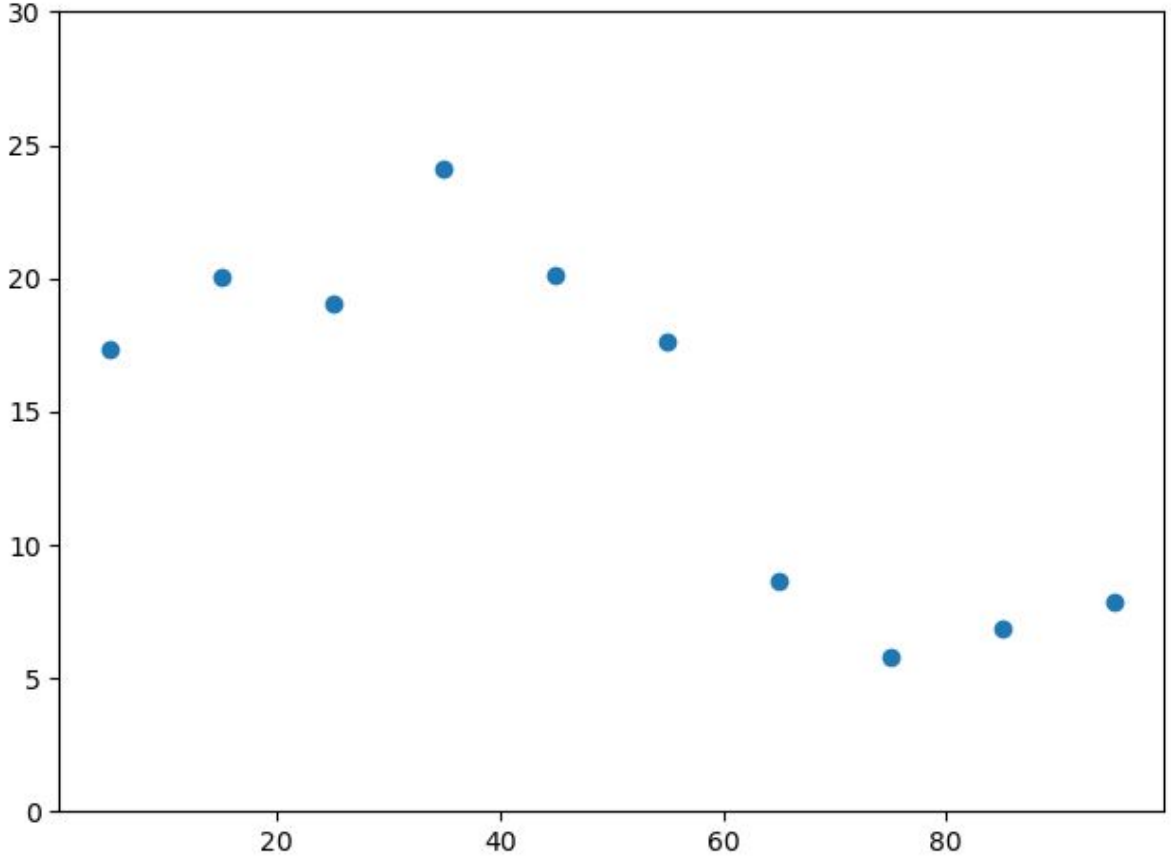


Supervised Learning - Goals

When performing supervised learning, our goal is to minimize the expected loss over *new* data.

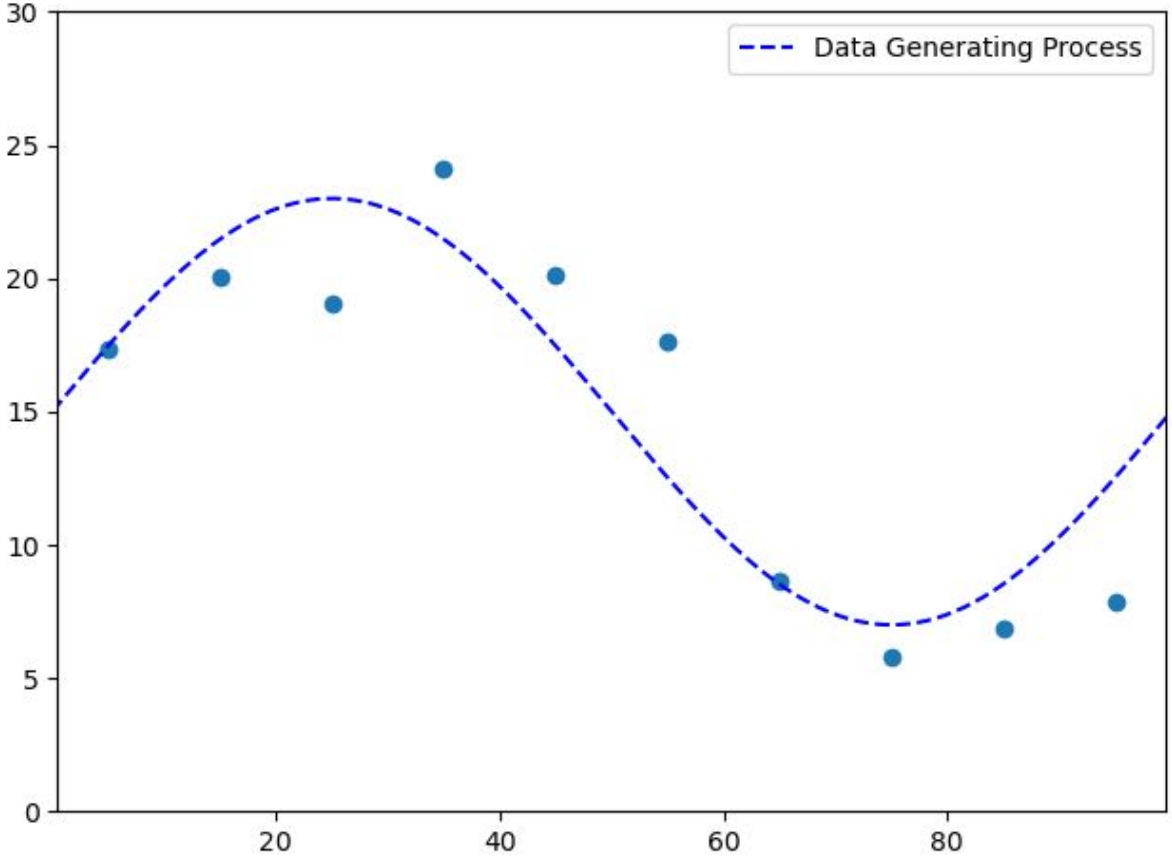
Important: We are not trying to minimize loss over the observed data (which is often very easy to do), but to minimize the *generalization error* - the performance on unseen data.

Example - Noisey Sine



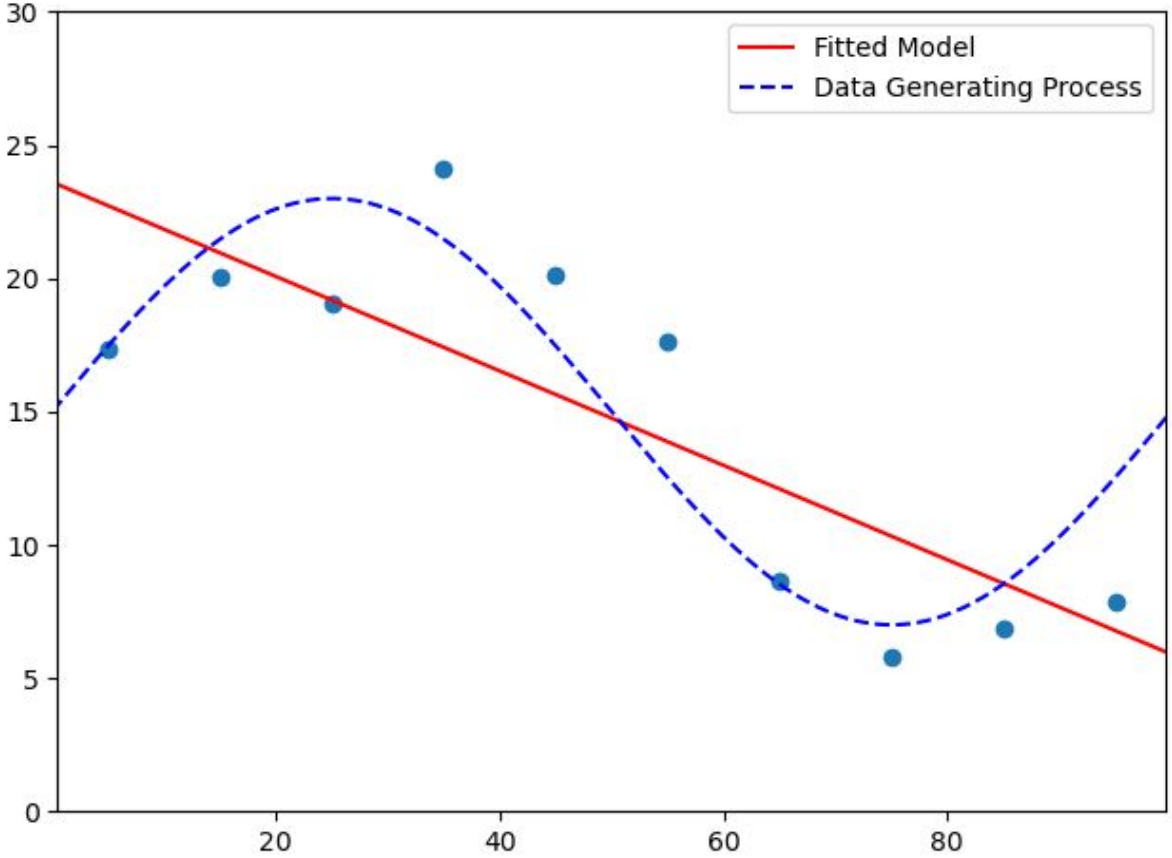
Example: Let's say we want to fit a model to predict the y-variable from this dataset.

Example - Noisey Sine



In this case, let's say we know the data generating process - a sine function plus some random noise.

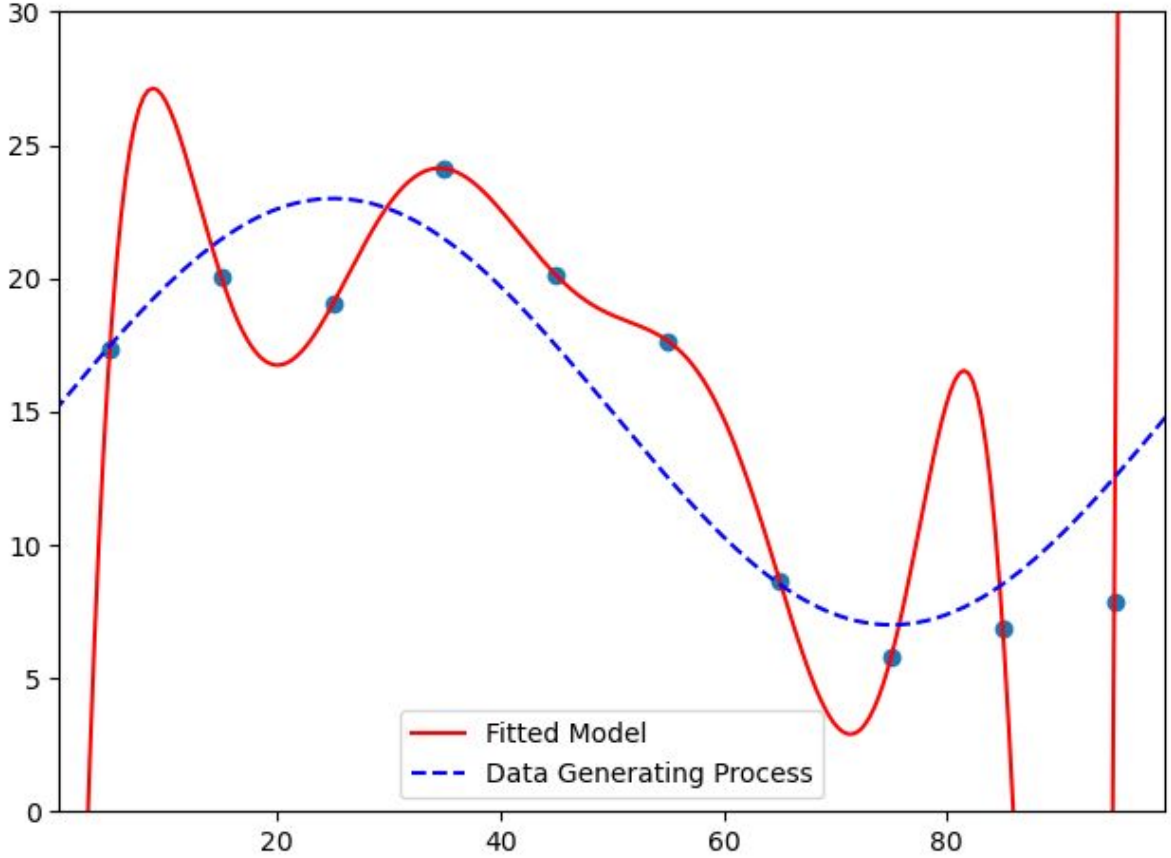
Example - Noisey Sine



We could use a simple linear regression model.

However, this does a poor job on both the training data and the actual function. It is **underfitting.**

Example - Noisey Sine



If we let our function be more flexible, we can perfectly predict the training data but do a terrible job at estimating the true function. We are **overfitting**.

Moral

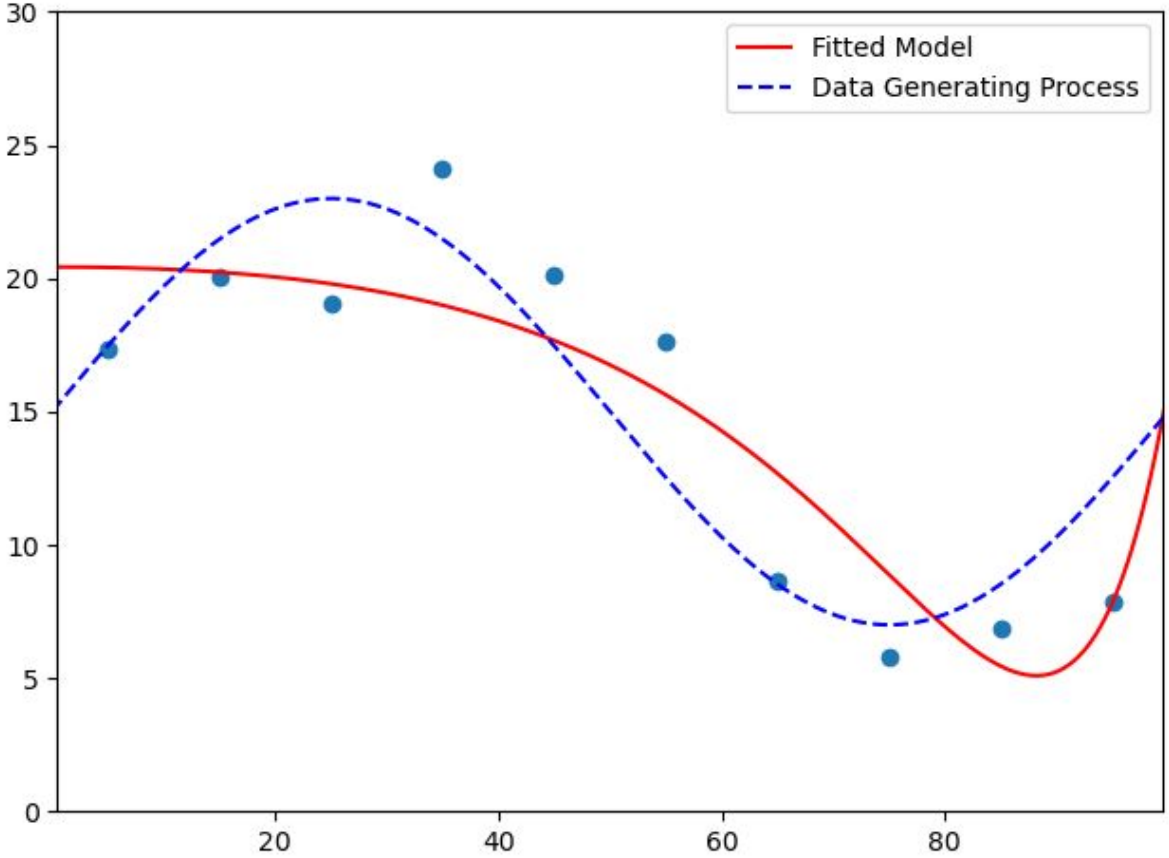
The goal of supervised learning is to find a model which has low generalization error.

Flexible models can be quite good at fitting the training data, but can do very poorly at generalizing to unseen data.

One strategy to avoid overfitting is to choose a simpler model.

A way to force a simpler model is **regularization**.

Example - Noisey Sine



If we apply regularization - here ridge regression - we can get a result that will have better generalization error.

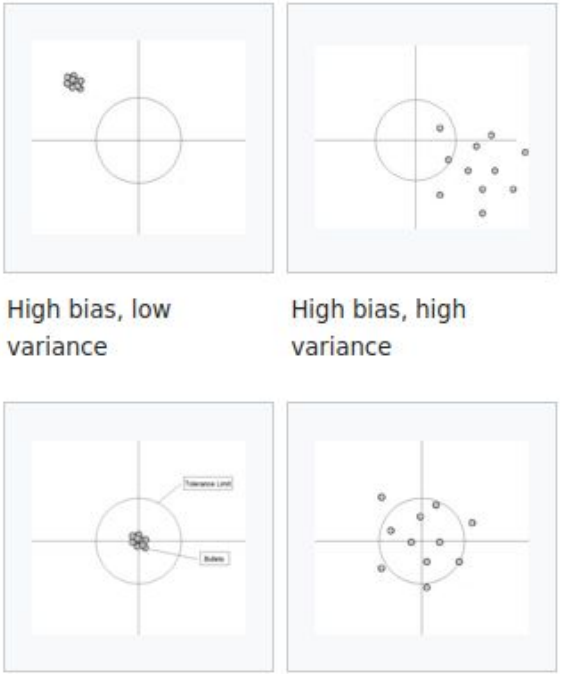
Bias-Variance and Regularization

The Bias-Variance Decomposition

Our generalization error can be decomposed into **bias** and **variance**.

A model with high **bias** means that it's expected value will be different from the true average value of the target. High bias lead to underfitting.

A model with high **variance** will be sensitive to changes in the training data. High variance leads to overfitting.



https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff

The Bias-Variance Decomposition

Regularization allows us to trade some bias for less variance.

Ridge Regression

Recall that linear regression fits a model of the form

$$\hat{f}(\vec{x}) = \beta_0 + \beta_1 x^{(1)} + \beta_2 x^{(2)} + \cdots + \beta_k x^{(k)}$$

By minimizing the residual sum of squares:

$$RSS = \sum_{i=1}^{n} (y_i - \hat{f}(\vec{x}_i))^2$$

Ridge Regression

Ridge regression also fits a model of the same form:

$$\hat{f}(\vec{x}) = \beta_0 + \beta_1 x^{(1)} + \beta_2 x^{(2)} + \cdots + \beta_k x^{(k)}$$

But seeks to minimize a different objective function:

$$L = \sum_{i=1}^{n} (y_i - \hat{f}(\vec{x}_i))^2 + \lambda \sum_{i=1}^{k} \beta_k^2$$

Ridge Regression

Want small residuals

$$L = \boxed{\sum_{i=1}^{n} (y_i - \hat{f}(\vec{x}_i))^2} + \lambda \sum_{i=1}^{k} \beta_k^2$$

Ridge Regression

Want small residuals

But keep the coefficients small

$$L = \boxed{\sum_{i=1}^{n} (y_i - \hat{f}(\vec{x}_i))^2} + \boxed{\lambda \sum_{i=1}^{k} \beta_k^2}$$

LASSO Regression

LASSO regression also fits a model of the same form:

$$\hat{f}(\vec{x}) = \beta_0 + \beta_1 x^{(1)} + \beta_2 x^{(2)} + \cdots + \beta_k x^{(k)}$$

But seeks to minimize a different objective function:

$$L = \sum_{i=1}^{n} (y_i - \hat{f}(\vec{x}_i))^2 + \lambda \sum_{i=1}^{k} |\beta_k|$$

LASSO Regression

LASSO regression is useful for regularization.

It also has the benefit that it can perform *feature selection*. Because math, LASSO regression has a tendency to set some of the coefficients equal to 0.

Logistic Regression - Regularization

We can also constrain the coefficients of a logistic regression model in a similar way using Ridge or LASSO penalties.

Validating the Model

The loss function for lasso and ridge included a parameter, lambda. How do we know which value of lambda to use?

An option is to try out several different values for lambda and see which performs best.

However, if we choose the best value on the test set, we no longer have an unbiased estimate of how well the model will do.

A better approach is to use **cross-validation**.

k-Fold Cross-Validation

Full Dataset

First, split into a train
and a test set.

k-Fold Cross-Validation

k-Fold Cross-Validation

Then, divide the training data into k (here 5) equally-sized pieces.

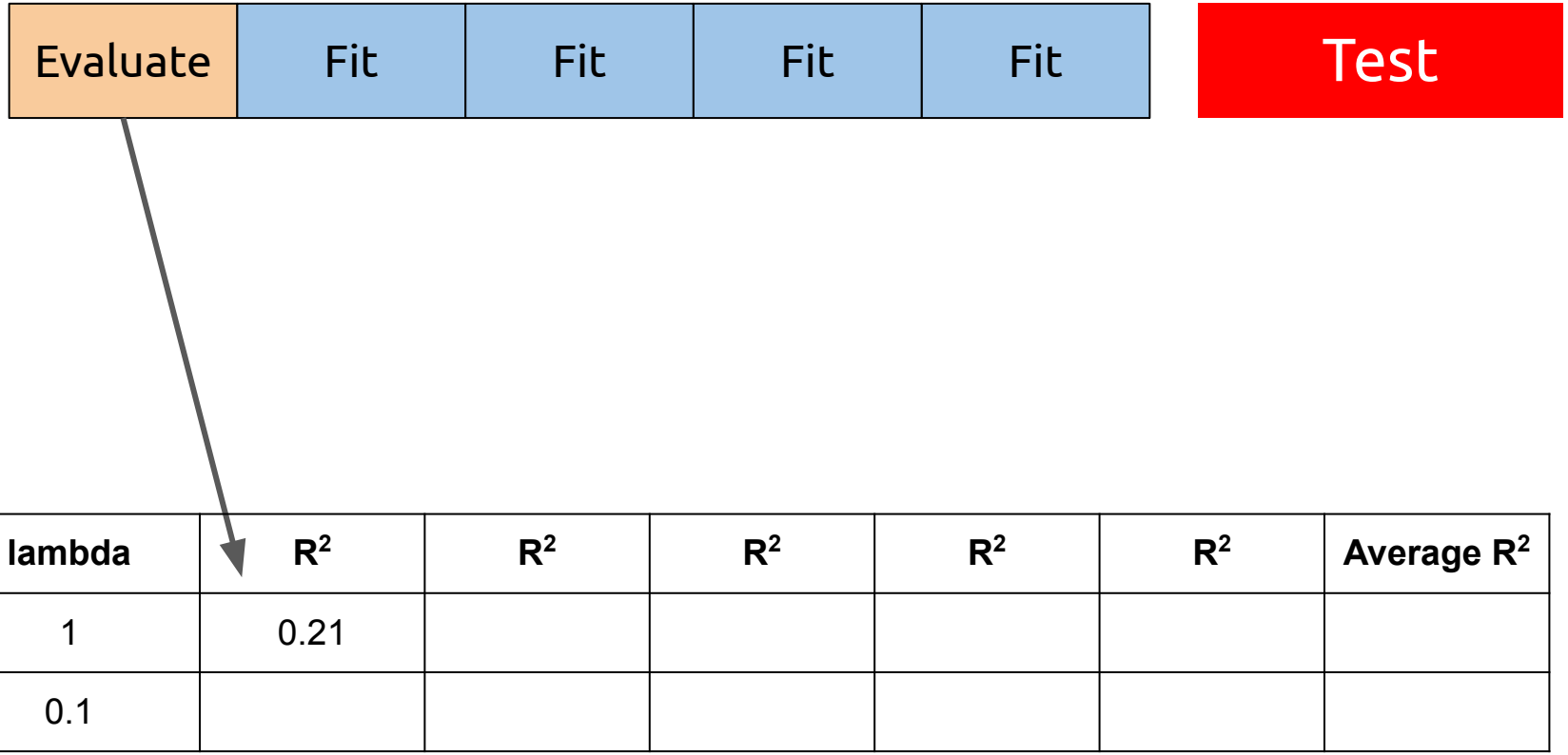
k-Fold Cross-Validation

For each value of the hyperparameter(s), fit k different models, each on k-1 of the pieces of the training data and measure the performance on the remaining one piece.

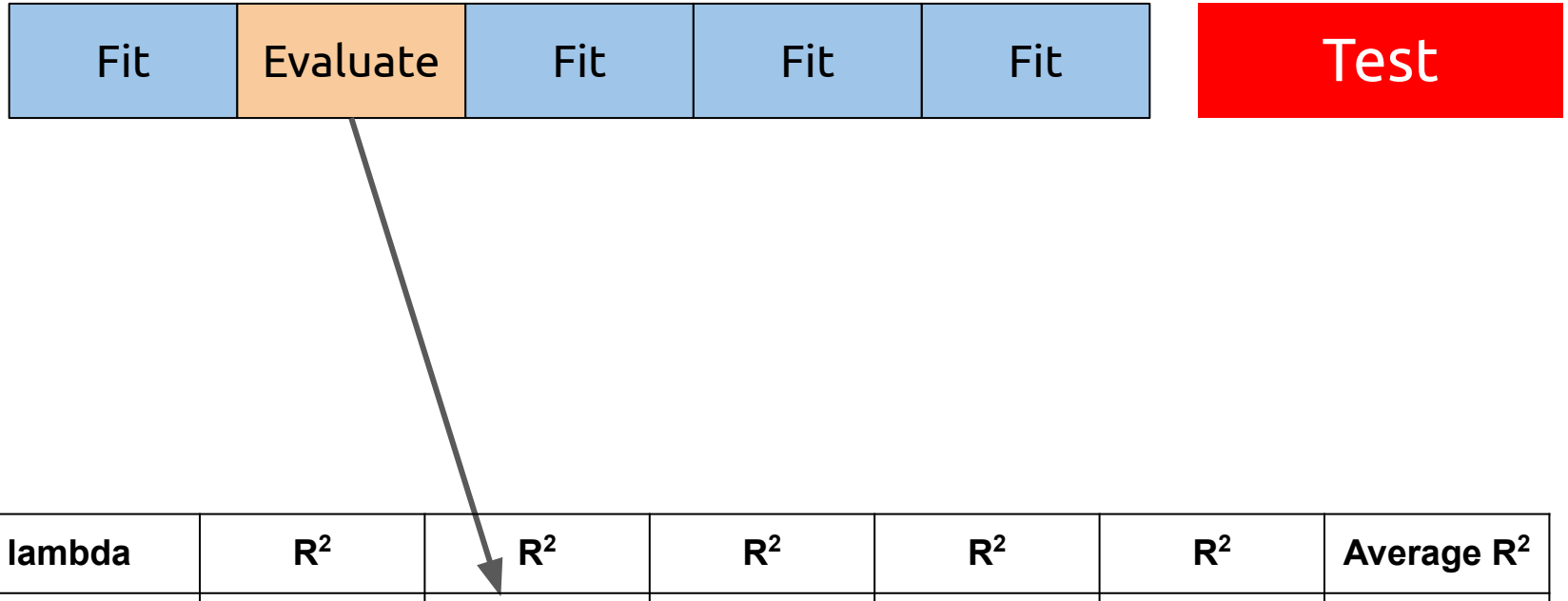
k-Fold Cross-Validation

lambda	R^2	R^2	R^2	R^2	R^2	Average R^2
1						
0.1						

k-Fold Cross-Validation

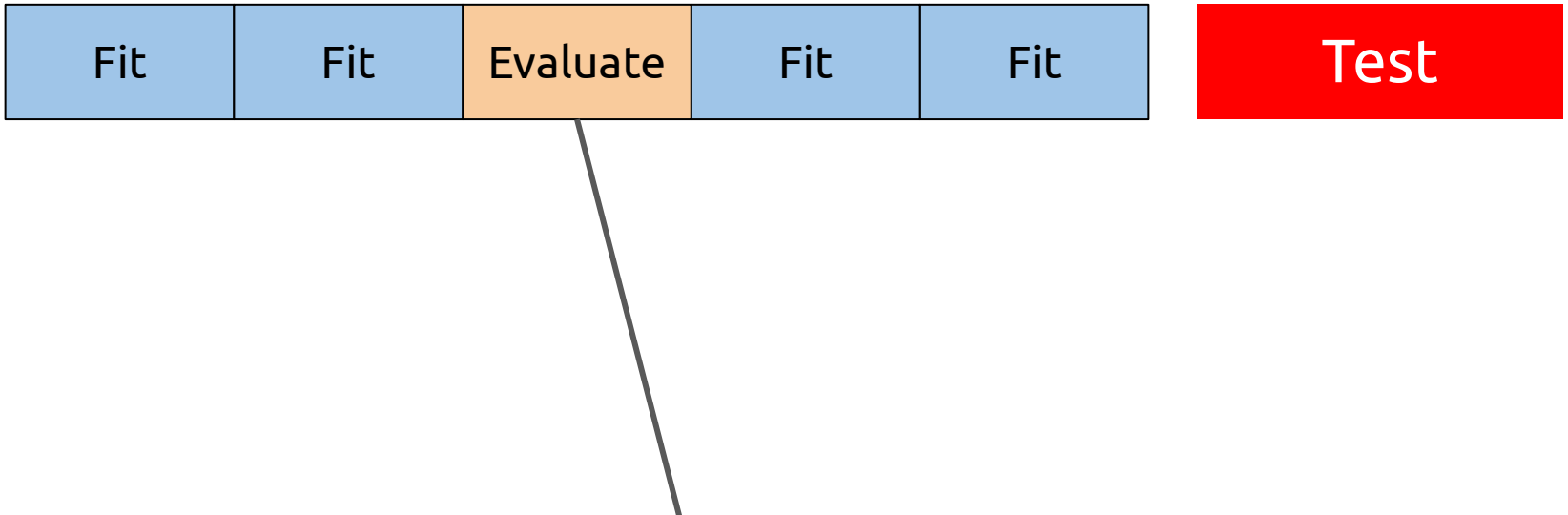


k-Fold Cross-Validation



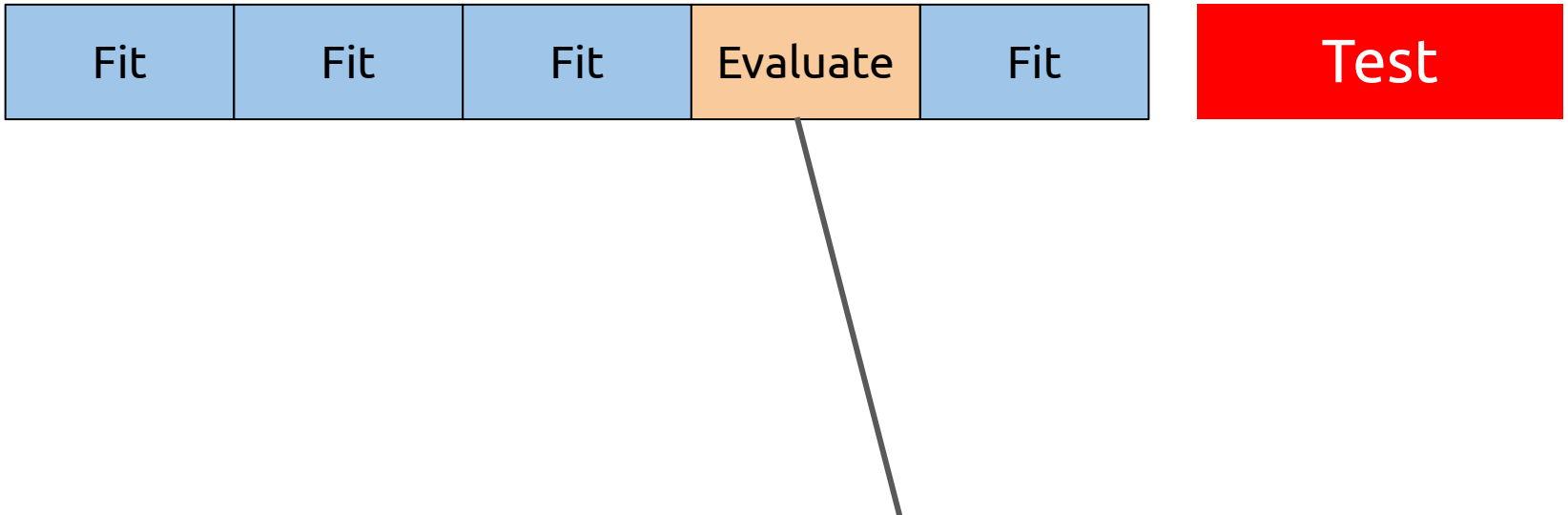
lambda	R^2	R^2	R^2	R^2	R^2	Average R^2
1	0.21	0.58				
0.1						

k-Fold Cross-Validation



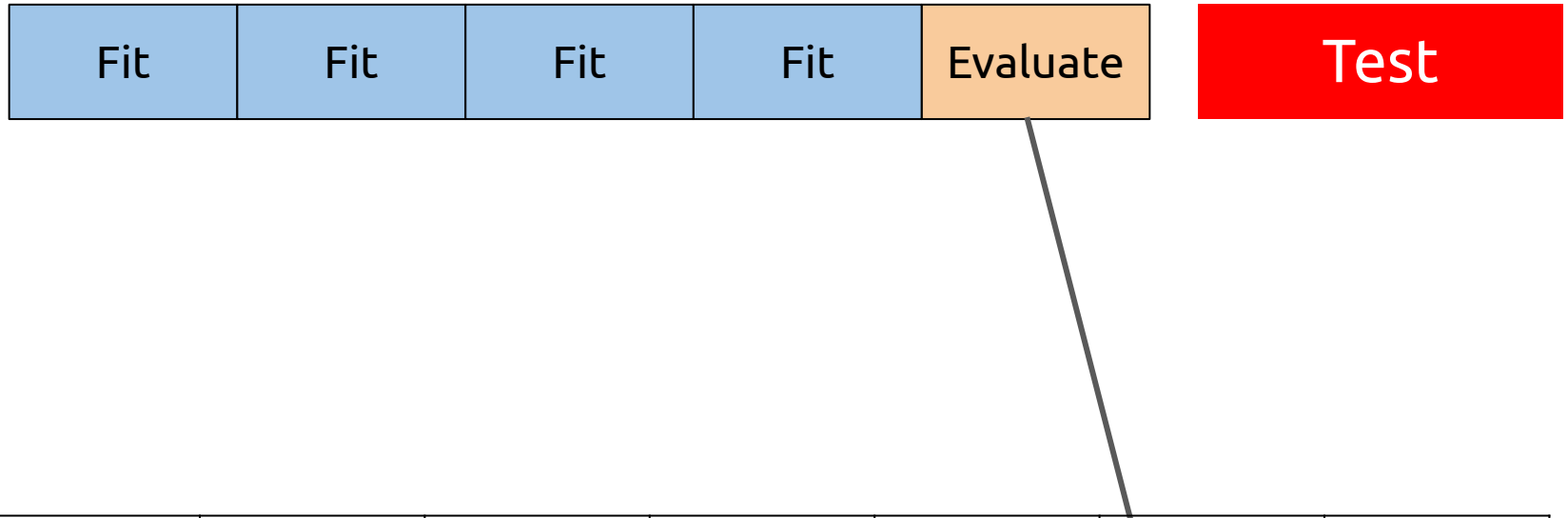
lambda	R^2	R^2	R^2	R^2	R^2	Average R^2
1	0.21	0.58	0.05			
0.1						

k-Fold Cross-Validation



lambda	R^2	R^2	R^2	R^2	R^2	Average R^2
1	0.21	0.58	0.05	0.56		
0.1						

k-Fold Cross-Validation



lambda	R^2	R^2	R^2	R^2	R^2	Average R^2
1	0.21	0.58	0.05	0.56	0.47	
0.1						

k-Fold Cross-Validation

Then average the results together.

lambda	R^2	R^2	R^2	R^2	R^2	Average R^2
1	0.21	0.58	0.05	0.56	0.47	0.37
0.1	0.28	0.62	-0.1	0.47	0.48	0.35

k-Fold Cross-Validation

Repeat for other values of your hyperparameter(s).

lambda	R^2	R^2	R^2	R^2	R^2	Average R^2
1	0.21	0.58	0.05	0.56	0.47	0.37
0.1	0.28	0.62	-0.1	0.47	0.48	0.35

k-Fold Cross-Validation

Choose the option with the best average performance.

To get a final estimate of performance, evaluate on the held-out test data.

lambda	R^2	R^2	R^2	R^2	R^2	Average R^2
1	0.21	0.58	0.05	0.56	0.47	0.37
0.1	0.28	0.62	-0.1	0.47	0.48	0.35